VLS 731 Proposal: Implementation of a Dot-Product-Dual-Accumulate Architecture for Transprecision Computing

Unnath Chittimalla (IMT2023620)
Yash Anand (IMT2023624)
Snehal Sharma (IMT2023572)
Tanmay Kumar (IMT2023593)
Rajan Samar (IMT2023616)
Nipun Verma (IMT2023591)

Project Topic

This project falls under the topic of **Floating-Point Arithmetic Unit Design and Processor Microarchitecture** within VLSI Architecture Design. Specifically, it focuses on **transprecision and mixed-precision floating-point architectures** for HPC-enabled AI workloads, based on the proposed Dot-Product-Dual-Accumulate (DPDAC) architecture.

Background and Motivation

Modern HPC and AI systems require efficient support for multiple floating-point formats such as DP, SP, TF32, HP, and BF16. Conventional architectures rely on discrete FMA and adder units, resulting in increased latency, area, and power overhead. The selected paper proposes a unified DPDAC architecture that natively supports multi-term dot-products, accumulation, dual-path execution, and numerical precision conversion within a single datapath, significantly improving hardware efficiency.

Related Concepts

The project directly involves core concepts from the course, including:

- IEEE-754 floating-point formats and datapath design
- Fused Multiply-Add (FMA) and dot-product architectures
- Multi-term accumulation using carry-save adders
- Mixed-precision and transprecision computing
- Pipeline architecture and latency optimization
- Alignment shifting, normalization, and rounding
- Area–performance trade-offs in multiplier design

Proposed Implementation

As part of this course project, we propose to **implement the DPDAC architecture as described in the paper without simplification**. The scope of implementation includes:

1. Full support for DPDAC operations ($\sum A_i \times B_i + \sum C_j$) with single-path and dual-path execution.
2. Multi-format floating-point support including DP, SP, TF32, HP, and BF16.
3. Iterative multi-precision multiplier array as proposed, including segmentation and hardware reuse.
4. Complete alignment, carry-save accumulation, normalization, rounding, and exponent processing pipeline.
5. Integrated numerical precision conversion (NPC) within the datapath.
6. Structural Verilog HDL implementation closely following the paper's architectural blocks and pipeline stages.
7. Functional verification using directed and randomized testbenches, and synthesis-based evaluation of area and timing.

A Low-Cost Floating-Point Dot-Product-Dual-Accumulate Architecture for HPC-Enabled AI

Hongbing Tan, Libo Huang, Zhong Zheng, Hui Guo, Qianmin Yang, Li Shen, Gang Chen, Liquan Xiao, and Nong Xiao, *Senior Member, IEEE*

***Abstract*—The dot-product $\sum_{i=1}^{N} A_i \times B_i$ is one of the most frequently used operations for a wide variety of high-performance computing (HPC) and artificial intelligence (AI) applications. However, for large-scale algorithms, such as GEMM and FFT, independent additions are necessary to accumulate the results of length-limited dot-product in order to form the final result, thus increasing latency and overhead. Hence, we proposed a dot-product-dual-accumulate (DPDAC) architecture capable of performing ($\sum_{i=1}^{N=1,2,4} A_i \times B_i + \sum_{j=1}^{M=1,2} C_j$) on a wide range of formats. The proposed architecture supports both single-path and dual-path execution. The single path is designed for performing DP FMA or DPDAC of lower formats, while dual-path supports parallel operations for single-precision (SP) addition and 2-term SP or TF32 dot-product or 4-term HP or BF16 dot-product. Moreover, numerical precision conversion is also supported by the proposed architecture, allowing for the conversion of numbers to higher or lower formats. The proposed DPDAC has been demonstrated to significantly reduce the overhead in comparison to discrete designs that utilize multiple single-mode FP units to achieve the same functionalities. Furthermore, when compared to the state-of-the-art multiple-precision designs, the proposed architecture has been shown to support a wide range of formats and a greater variety of operations with lower costs.**

***Index Terms*—Dot-product-dual-accumulate (DPDAC), fused multiply-add, high-performance computing (HPC)-enabled artificial intelligence (AI), mixed-precision, numerical precision conversion, transprecision computing.**

I. Introduction

RECENTLY, the integration of high-performance computing (HPC) and artificial intelligence (AI) has become an exciting and innovative paradigm for academia and industry [1], [2], [3]. HPC-enabled AI workloads require sophisticated hardware support to process a wide range of data formats and perform trillions of calculations per second, or more. However, the versatility of data types and the tremendous computational power of HPC and AI converged computing lead to expensive costs and complexity in hardware design. As a solution, transprecision computing [4] has been extensively explored to address the growing need for improved performance and energy efficiency in modern computing systems.

In addition to the commonly used double-precision (DP) and single-precision (SP) formats in traditional HPC systems, recent research efforts have explored various formats in modern systems to achieve higher energy efficiency. For example, Intel Nervana's Flexpoint [5], Google TPU's 16-bit "Bfloats (BF16)" [6], and NVIDIA's 19-bit TestFloat-32 (TF32) implemented in specific accelerators such as Tensor Cores [7], demonstrate the necessity of new architectures with extreme transprecision flexibility for floating-point (FP) computation. These advancements, driven by deep learning algorithms and applications, highlight the growing demand for more efficient FP operations. Compared to high-precision formats, lower precision numbers can not only save memory resources but also be implemented more efficiently in hardware due to their smaller bitwidth. However, the implementation of HPC-enabled AI algorithms using complete low-precision operations may incur accuracy loss due to its narrower dynamic range or limited precision [8], [9]. Thus, mixed-precision methods are also widely explored to increase performance while maintaining high accuracy, performing multiplications in lower precision while accumulating products into higher precision. For example, in the field of linear algebra, SP [10] and half-precision (HP) [8] are used as the low-precision formats for the majority of computations in iterative refinement, while DP is used at the last few stages to refine the results. Likewise, in order to reduce the accumulated rounding error of deep learning algorithms, low-precision formats, such as TF32 [11], HP [9], and BF16 [12] multiplications need to be accumulated into SP format in convolution (CONV) or general matrix multiplication (GEMM) of deep learning applications.

So far, transprecision FP multiply-accumulate (MAC) units have been developed to reduce area, power consumption, latency, and improve accuracy compared to a network of discrete FP units by executing common logic for multiple operations in parallel. Previous FP units in [13], [14], and [15] support multiple formats and improve throughput by using parallel operations, but have lower efficiency in performing

Manuscript received 21 February 2023; revised 6 July 2023; accepted 27 August 2023. Date of publication 18 September 2023; date of current version 22 January 2024. This work was supported by the Key Laboratory of Advanced Microprocessor Chips and Systems; in part by NSFC under Grant 62272475 and Grant 62090023; and in part by NSFHN under Grant 2022JJ10064. This article was recommended by Associate Editor K. Olcoz. *(Corresponding author: Libo Huang.)*

Hongbing Tan, Libo Huang, Zhong Zheng, Hui Guo, Qianmin Yang, Li Shen, and Liquan Xiao are with the College of Computer Science and Technology, National University of Defense Technology, Changsha 410073, China (e-mail: libohuang@nudt.edu.cn).

Gang Chen and Nong Xiao are with the School of Data and Computer Science, Sun Yat-sen University, Guangzhou 510006, China.

Digital Object Identifier 10.1109/TCAD.2023.3316994

1937-4151 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on January 27,2026 at 12:38:41 UTC from IEEE Xplore. Restrictions apply.

dot-product algorithms, such as fast Fourier transform (FFT), GEMM, and CONV. Designs in [16] and [17] proposed FP dot-product units that sum up multiple products together to improve performance and reduce overhead. However, when using these dot-product units to construct the computing systems, independent adders are required to sum up the result generated by each unit, resulting in extra hardware costs. Additionally, previous FP units primarily focused on the implementation of HPC applications, but provided limited support for transprecision operations that are well-suited for deep learning algorithms.

Hence, we proposed a dot-product-dual-accumulate (DPDAC) architecture capable of supporting transprecision FP operations for both HPC and AI algorithms. The proposed architecture supports both single-path and dual-path execution by splitting the DPDAC into two parts for a broad range of scenarios. For single-path execution, it can perform one DP fused multiply-accumulate (FMA) or one DPDAC of lower formats. For dual-path execution, it supports parallel operations for SP addition and 2-term SP or TF32 dot-product or 4-term HP or BF16 dot-product. In addition to supporting multiply-add operations, the DPDAC architecture can be modified to facilitate numerical precision conversion, thereby enabling the conversion of numbers to higher or lower formats to accommodate the datatype transformation in the datapath of mixed-precision algorithms. The proposed DPDAC significantly reduces the overhead compared to the discrete design that utilizes multiple single-mode FP units to achieve the same functionalities. Furthermore, compared to the state-of-the-art multiple-precision designs, the proposed architecture supports a wide range of formats and more types of operations, such as DPDAC, *NPCv*, and dual-path execution, with lower costs.

The major contributions of this article are summarized as follows.

1) We propose a DPDAC architecture capable of supporting $(\sum_{i=1}^{N=1,2,4} A_i \times B_i + \sum_{j=1}^{M=1,2} C_j)$. The proposed architecture can be configured to perform dot-product, addition, and FMA by setting the operational modes.
2) The proposed architecture supports both single-path and dual-path execution for transprecision FP operations, allowing for one DP FMA or one DPDAC of lower formats to be performed in single-path execution, as well as parallel operations for SP addition and 2-term SP or TF32 dot-product or 4-term HP or BF16 dot-product in dual-path execution.
3) The proposed architecture supports *NPCv* in mixed-precision operations. Numbers can be converted to higher or lower formats based on the practical requirement of mixed-precision algorithms with negligible overhead.
4) The proposed DPDAC architecture supports more formats and functionalities compared to other multiformat FP units with lower costs.

The remainder of this article is structured as follows: Section II provides the background for this work. Section III outlines the design of the proposed arithmetic architecture. Section IV details the circuit implementation of each module. Section V presents the methodology, synthesis results, analysis, and comparison with previous works. Finally, Section VI concludes the article.

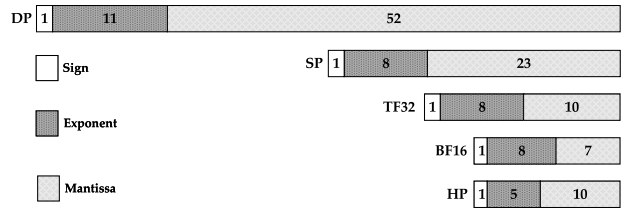


Fig. 1. Supported FP formats of the proposed DPDAC architecture.

II. Background

A. Numerical Format

The standard FP formats are defined in IEEE754-2008 standard [18] which consist of three components: 1) the 1-bit sign (S); 2) the e-bit biased exponent (E); and 3) the m-bit mantissa (M). The actual value of the normal number represented in IEEE 754-2008 format is

$$(-1)^S \times \left(im + 2^{-n} \times M\right) \times 2^{E-\text{bias}} \tag{1}$$

the bias is the exponent bias whose value is $2^{m-1} - 1$, im is the implicit bit for the mantissa. For normal numbers, $im = 1$. For zero and subnormal numbers, $im = 0$. In this article, the components of the standard formats, DP, SP, and HP, defined in IEEE 754-2008, and the TF32 and BF16 formats are shown in Fig. 1.

B. Conventional FP FMA Architecture

The basic FP FMA architecture that computes the operation $A \times B + C$ is implemented as the baseline design of the proposed DPDAC architecture. In conventional FP FMA architecture, all the operands are of the same precision, and the whole datapath is usually divided into three pipeline stages to improve performance. The datapath of the conventional FP FMA architecture is described as follows.

1) The mantissa padded with 1-bit implicit bit, *ManA* and *ManB*, are multiplied by an unsigned multiplier, and then generate the carry-save format product.
2) At the same time, the exponent difference between product $A \times B$ and C, the sign processing, the exponent prediction, and alignment for addend C based on the value of exponent difference are also performed in the first pipeline stage.
3) Then, in the second pipeline stage, the aligned addend C and the mantissa product are added together through a carry save adder (CSA) and a carry propagate adder (CPA), and the normalization shifting amount is also predicted by a leading zero anticipator and counting (LZAC) unit at the same time.
4) Finally, normalization shifting and exponent adjustment are performed according to the leading zero amount; and then rounding to generate the mantissa. The FMA result

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on January 27,2026 at 12:38:41 UTC from IEEE Xplore. Restrictions apply.

TABLE I
MAIN ABBREVIATIONS IN THIS ARTICLE

Parameter	Description
DPDAC	**D**ot-**P**roduct-**D**ual-**AC**cumulate
FP	**F**loating-**P**oint
FMA	**F**used **M**ultiply-**A**dd
MAC	**M**ultiply-**AC**cumulate
NPCv	**N**umerical **P**recision **C**onversion
ASC	**A**lignment **S**hifter **C**ount
MSB,LSB	**M**ost/**L**east **S**ignificant **B**it
CPA, CSA	**C**arry **S**ave **A**dder, **C**arry **P**ropagate **A**dder
LZAC	**L**eading **Z**ero **A**nticipator and **C**ounting

is obtained after the combination of the processed sign, the adjusted exponent, and the rounded mantissa.

In this article, we proposed a DPDAC architecture that modifies the conventional FP FMA architecture to support a wide range of formats and more operations. The main abbreviations used in this article are listed in Table I. Compared to the conventional FP FMA operations, the proposed architecture can be configured to perform FMA, addition, and dot-product by setting part of the operands to "0." The proposed architecture supports mixed-precision operations, where A and B are of lower precision while C is of higher precision, as well as dual-path execution, allowing two operations to be performed in parallel. Moreover, the *NPCv* are also merged into the datapath of the proposed DPDAC architecture, which can convert high-precision results to lower precision formats, and by setting C to "0" and B to "1" in mixed-precision operation, the lower precision operand A can be converted to higher precision results. The basic implementation steps of the proposed architecture are similar to the conventional FP FMA architecture, however, the bitwidth of the datapath is extended to accommodate the higher precision addend and the pipeline stages and the datapath need to be adjusted significantly to improve performance and realize the added functionalities for improved performance and efficiency.

C. DPDAC in Processors Architecture

The arithmetic units are designed and organized for constructing the computing architecture of processors to achieve high performance. Two mainstream computing architectures have been applied in previous processors that the systolic array architecture in [6] and [19] and the adder tree architecture in [20] and [21]. A typical 2-D systolic array used in TPU [6] is shown in Fig. 2(a). For the dataflow of the systolic array, each MAC cell receives data from the up and left cells (i.e., A and B) and the intermediate results (i.e., C) generated by the up cell. Before writing to the accumulator buffer, the results generated by the MAC cells of the bottom row must be added to the intermediate results from the accumulator buffer. Fig. 2(b) shows an adder tree architecture proposed by [21], in which the weights and activations from the W-Buffer and A-buffer are sent to the calculation pipelines to perform matrix

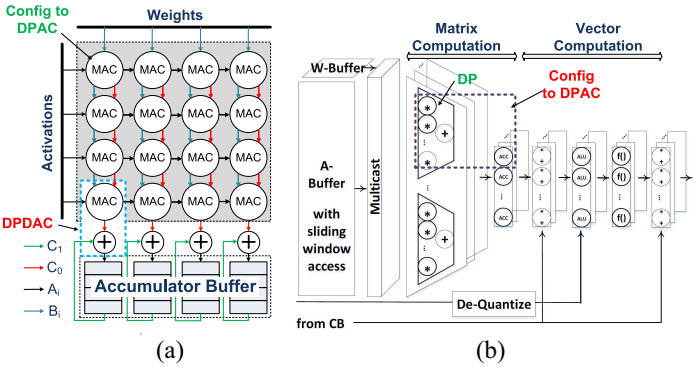


Fig. 2. DPDAC in previous computing architecture. (a) DPDAC in systolic array architecture. (b) DPDAC in adder tree architecture.

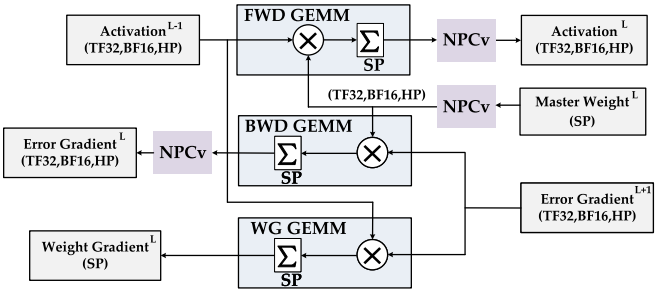


Fig. 3. Mixed precision dataflow used for DNN's training.

and vector computations. In matrix computation stages, dot-product is performed by adder tree, and the generated results are sent to an adder to be accumulated with the intermediate results from the convolution buffer (CB). As shown in Fig. 2, both the systolic array and adder tree architectures incorporate an adder to accumulate the results generated by MAC cells with the previous intermediate results. The proposed DPDAC architecture can be configured into various modes to meet the requirements of these two computing architectures.

Moreover, the DPDAC unit has the potential to be a fundamental building block in general-purpose architectures to efficiently perform deep learning instructions, such as the Intel's AVX-512 VNNI [22], Arm's v8-SVE [23], and v9-SME. With the support of mixed-precision dot-product, it has the capability to significantly reduce latency and enhance the accuracy of deep learning operations.

D. NPCv in Mixed-Precision Datapath

Mixed-precision computation is extensively employed in DNN's training [9], [12] and iteratively refining matrix equation solvers [8], [24] for enhanced performance and overhead reduction. The dataflow of mixed-precision training of DNNs, where the data type undergoes transformation in different layers, is illustrated in Fig. 3. The core computations of forward (FWD)/backward (BWD) propagation or weight gradient (WG) are represented as GEMM operations, which accept inputs as low-precision tensors and accumulate the output to SP tensors. In the FWD/BWD propagation, the low-precision activations or error gradients together with the converted weight are sent to the GEMM, and then the *NPCv* converts the

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on January 27,2026 at 12:38:41 UTC from IEEE Xplore. Restrictions apply.

GEMM output tensors to low-precision formats before passing them to the next layer.

The proposed DPDAC architecture offers three key benefits by adding the circuits for *NPCv*.

1) Compared to designing a standard *NPCv* cell, minor modifications to the output processing and rounding in the proposed architecture can achieve the same functionality.
2) The proposed architecture eliminates the need for storing high-precision GEMM results in memory and subsequent retrieval for conversion in an *NPCv* cell. This approach effectively reduces two memory accesses, thereby enhancing the efficiency of mixed-precision algorithms.
3) Storing the converted lower precision number in memory can effectively reduce the memory footprint by half.

III. DPDAC ARCHITECTURE

A. Functions of the DPDAC Architecture

The proposed DPDAC architecture supports a broad range of FP formats to meet the practical requirements of HPC and AI algorithms. Specifically, DP and SP formats are utilized for HPC applications, while SP, TF32, HP, and BF16 are suitable for deep learning scenarios. In addition to the standard and mixed-precision FMA operations similar to the pioneering designs of [25], [26], and [27], the proposed design also includes the newly added DPDAC and *NPCv*, as well as dual-path execution. The operations supported by the proposed design can be summarized.

1) Based on the practical requirements of HPC algorithms, the proposed design supports both DP and SP operations with the same precision. To improve performance while maintaining high accuracy for deep learning algorithms, mixed-precision operations are utilized, wherein lower precision products, such as TF32, HP, or BF16 formats, are accumulated to SP format.
2) The DPDAC ($\sum_{i=1}^{N=1,2,4} A_i \times B_i + \sum_{j=1}^{M=1,2} C_j$) is proposed and can be configured to perform both single-path and dual-path operations. One DP FMA or one DPDAC of lower formats can be performed in single-path execution, while parallel operations for SP addition and 2-term SP or TF32 dot-product or 4-term HP or BF16 dot-product are supported in dual-path execution.
3) The *NPCv* operations facilitate the conversion of numbers to higher or lower precision formats in mixed-precision algorithms. The low-to-high conversion is implemented in accordance with the dataflow of the proposed architecture, while high-to-low conversion is achieved by setting the value of operands in mixed-precision operations.

In the proposed design, the supported DPDAC can be leveraged to significantly improve the throughput and reduce rounding errors for practical HPC-enabled AI workloads. For instance, the FFT butterfly architecture [28] requires a 2-term dot-product, and more terms of products need to be accumulated in CONV and GEMM. Compared to the serial computations in FMA units, the throughput is significantly improved by accumulating the parallel multiplication products in one operation. Additionally, the rounding errors are also minimized due to the fewer rounding times.

B. Tradeoff Between Area and Performance in Multiplier Design

The proposed design supports DP, SP, TF32, HP, and BF16 formats in one architecture, constructed using high-precision-split methods. The design of the multiplier features a quadratic growth in silicon area with respect to the number of mantissa bits. For example, a 16-bit HP multiplier using 11 mantissa bits requires $11^2 = 121$ units of area, while a 32-bit SP multiplier considering 24 mantissa bits employs $24^2 = 576$. To reduce overhead, iteration methods are applied in the multiplier design, at the cost of performance. Based on the highest format supported by the proposed design, the number of iterations (N_{iter}) is utilized to make a tradeoff between area and performance for the multiplier array. Thus, the number of units (N_{unit}) required for constructing the multiplier array is inversely proportional to N_{iter}

$$N_{\text{unit}} = \left\lceil W_{\text{DP}}^2 / N_{\text{iter}} \right\rceil \tag{2}$$

where W_{DP} denote the mantissa bits of the DP format. The number of parallel operations N_P for each format is limited by two factors: 1) the number of units N_{unit} and 2) the bitwidth of the input operands. This can be expressed as follows:

$$N_P = \max\left(\min\left(\left\lfloor \frac{64}{BW_P} \right\rfloor, \left\lfloor \frac{N_{\text{unit}}}{W_P^2} \right\rfloor\right), 1/\left\lceil \frac{W_P^2}{N_{\text{unit}}} \right\rceil\right) \tag{3}$$

where BW_P and W_P are the bitwidth and mantissa bits of the aimed formats, respectively. The constant 64 is the bitwidth of the input operands. Various formats are used in transprecision computing to achieve high efficiency in practical HPC and AI converged algorithms. The parallel operations of formats, N_P, is thus related to the performance of the practical workloads. Assuming that R_P is the ratio of computation of the whole algorithms of the dedicated format, the runtime T can be calculated as follows:

$$T = \left(\sum_{P \in (DP, SP, TF32, HP, BF16)} \frac{R_P}{N_P}\right). \tag{4}$$

Based on the runtime, we obtained the unified performance P_u as follows:

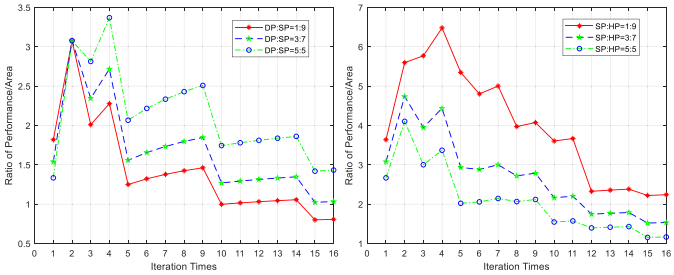
$$P_u = 1/T. \tag{5}$$

By using iterative methods, the area of the multiplier can be reduced by a factor of the number of iterations. This, in turn, reduces the area of the entire design in proportion to the area of the multiplier. The ratio of the multiplier area in the whole design is denoted as R_{mul}, then we calculate the area (A) in a scaled format as follows:

$$A = R_{\text{mul}}/N_{\text{iter}} + (1 - R_{\text{mul}}). \tag{6}$$

By increasing the iteration times, we can reduce the area of the proposed design at the cost of performance. We define

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on January 27,2026 at 12:38:41 UTC from IEEE Xplore. Restrictions apply.



(a) (b)

Fig. 4. Performance/Area under different iteration times in HPC and AI field. (a) DP and SP operations in HPC filed. (b) SP and HP operations in AI filed.

an index, P_A, to evaluate the methods based on the ratio of performance and area

$$P_A = \frac{P_u}{A}. \tag{7}$$

In the proposed design, the iteration times are determined by the highest precision format, i.e., DP format, which constrains the throughput for SP, TF32, HP, and BF16 operations. To evaluate the iteration times, we select typical scenarios using DP and SP operations in the HPC field and SP and HP operations in the AI field. We analyze the iteration times with performance/area by adjusting the ratio of these two formats and plot the results in Fig. 4. We set the R_{mul} to 0.7, which is approximately the ratio of the multiplier area in the conventional FP FMA unit. As illustrated in Fig. 4, we determined that two iteration times is the optimal choice for various scenarios.

C. Dataflow of the DPDAC Architecture

The proposed DPDAC architecture utilizes segmentation hardware vectorization methods to optimize performance, such as throughput and latency, while minimizing area and power consumption. The logic is divided into four parts based on the bitwidth of input operands to meet the demands of parallel computing. To manage the segmentation of hardware, three control modes (*PD_Mode*, *PD2_Mode*, *PD4_Mode*) are employed, which are determined by the number of parallel products.

The diagram of the proposed DPDAC architecture, illustrating the details of the proposed DPDAC datapath, is shown in Fig. 5. The whole datapath consists of four pipeline stages, each corresponding to a different stage of the extended functionalities. In the first stage, processing for the input data is implemented, wherein the components of the operands are checked, extracted, and rearranged. The multiple-precision multiplier, the logic for processing the sign and exponent, and the alignment shifter with its corresponding shifting control logic are also included. The second pipeline stage contains the logic for adding the products generated by the parallel multiplication operations in the first pipeline stage. The adder and the alignment shifter are used to align the products, and then generate the carry-save format products. The third pipeline stage merges the product of $A \times B$ and the aligned C operands

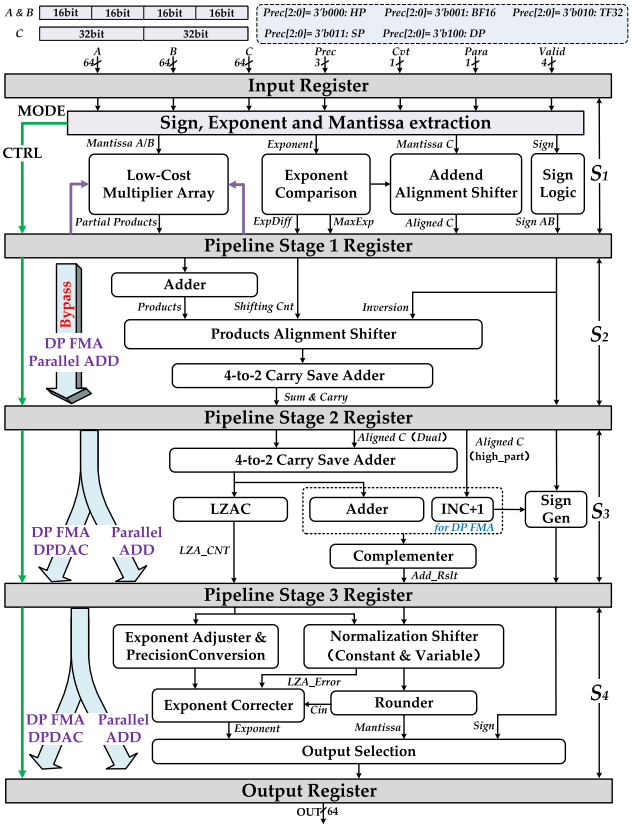


Fig. 5. Datapath of the proposed arithmetic architecture.

through a 4-to-2 CSA and adders. The calculation for the complement of the addition result and the prediction for the leading zero numbers by using LZAC are also performed in parallel. At the last pipeline stage, the logic for normalization, exponent adjustment, rounding, *NPCv*, and output result processing are contained.

The proposed DPDAC utilizes low-power techniques, such as gating the clock in the second pipeline stage for DP operations to reduce energy costs, resulting in a latency of three clock cycles compared to four clock cycles for other operations. Additionally, the multiplier array can be gated in low-precision multiplications. Furthermore, if input operands are invalid, the values of the pipeline registers are retained, thus avoiding the toggle of Flip-Flops and reducing power consumption.

The design is controlled by a 3-bit signal, *Prec*, supporting five formats (DP, SP, HP, TF32, and BF16). A 4-bit signal, *Valid*, enables the splitting of 64-bit operands into four 16-bit sections, while a 1-bit signal, *Cvt*, enables the *NPCv*. Furthermore, the *Para* signal regulates single-path and dual-path execution.

D. Steps of the DPDAC Implementation

The details of the implementation of the simplified DPDAC are presented in Algorithm 1, which comprises nine steps corresponding to the datapath of the proposed architecture. In addition to the function logic, a clock counter (*Cnt*) is designed to calculate the cycles and control the data selection

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on January 27,2026 at 12:38:41 UTC from IEEE Xplore. Restrictions apply.

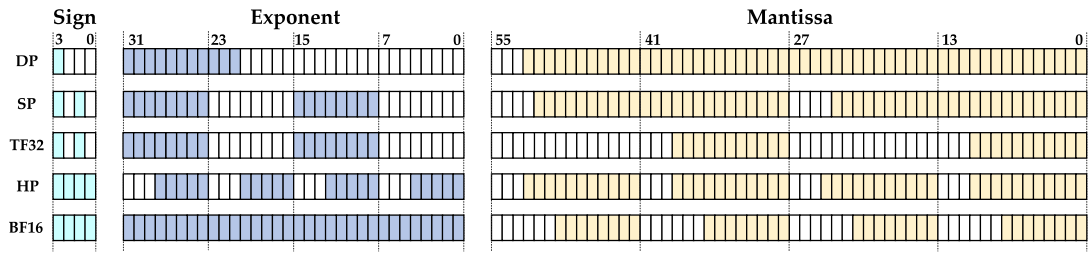


Fig. 6. Vector data formats for input vectors. White blocks represent unused bits (set to "0").

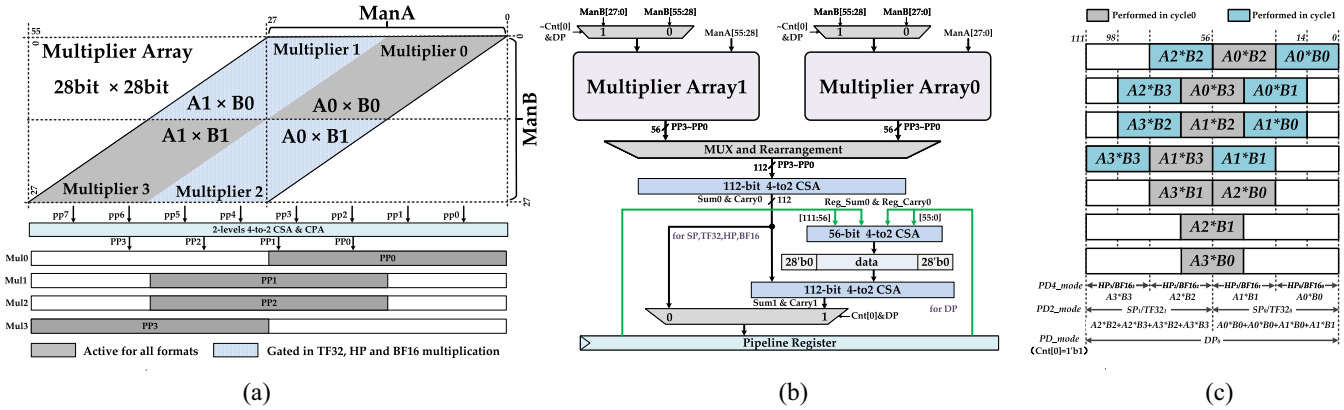


Fig. 7. Construct of the proposed multiple-precision multiplier array. (a) Organization of the multiplier array. (b) Dataflow of the multiple-precision multiplier array. (c) Active multiplier in different modes.

of the multiplier array. As illustrated in Algorithm 1, steps 1 to 5 are modified significantly to support DPDAC, while steps 6 to 8 are similar to a conventional FP FMA implementation. Steps 1 to 3 are executed in parallel in pipeline stage 1 to reduce latency, and the alignment shifting in step 4 is placed in pipeline stage 1 and pipeline stage 2 for product shifting and addend shifting, respectively. The dual-path addition in step 5 is located in pipeline stage 3 to support parallel operations for dot-product and addition by reusing the adder. In steps 6 to 8, there are minor modifications in bitwidth and datapath compared to a conventional FP FMA implementation. Step 9 merges the *NPCv* and, using a conventional method, obtains the exponent before subtracting the base value of the exponent difference between the higher and lower formats. Finally, the processed sign, exponent, and mantissa are combined to generate the final result.

IV. Circuit Implementation of the Proposed DPDAC Architecture

A. Input Operands Processing

To support multiple-precision operations in the proposed DPDAC architecture, the 64-bit input operands are decoded into three unified vector formats that gather the sign, exponent, and mantissa components for each supported vector element format. As illustrated in Fig. 6, each operand of the DPDAC corresponds to a 92-bit vector format, comprising a 4-bit sign vector, a 32-bit exponent vector, and a 56-bit mantissa vector. In the adopted formats, bits that are unnecessary to represent vector element values are set to "0." Each vector consists of four segments, with segment 0 and segment 2 only used for *PD*4_mode, segment 1 used for both *PD*4_mode and *PD*2_mode, and segment 3 applicable for all modes. To reduce power consumption, these unified vector formats are enabled by a 4-bit signal *Valid*. If one bit in *Valid* is zero, the corresponding segment maintains its previous value to avoid invalid toggle of flip-flops.

B. Floating-Point Multiple-Precision Multiplier Array

The proposed multiplier array performs the multiplication of the unified mantissa vectors, enabling multiple-precision multiplication for the supported modes. To reduce the hardware cost of the proposed DPDAC architecture, iteration methods are employed to tradeoff performance and area of the proposed multiplier array. Consequently, DP multiplication is implemented every other cycle utilizing recursive methods, while other lower precision multiplications are executed in a pipelined fashion. As illustrated in Fig. 7, the mantissa of the operands A and B have been organized into 56-bit unified vectors and split into four segments to meet the parallel requirements of the supported modes. The mantissa multiplication for different modes can be represented as follows:

$$\begin{aligned} PD_{\text{mode}} &: \{a_1, a_0\} \times \{b_1, b_0\} + \{\{a_3, a_2\} \times \{b_3, b_2\}\} << 56 \\ &\quad + \{\{a_1, a_0\} \times \{b_3, b_2\} + \{a_3, a_2\} \times \{b_1, b_0\}\} << 28 \\ PD2_{\text{mode}} &: \{\{a_3, a_2\} \times \{b_3, b_2\}, \{a_1, a_0\} \times \{b_1, b_0\}\} \\ PD4_{\text{mode}} &: \{a_3 \times b_3, a_2 \times b_2, a_1 \times b_1, a_0 \times b_0\} \end{aligned} \tag{8}$$

where the a_i and b_i refer to one 14-bit segment in the unified vectors. Based on (8), a 14×14 multiplier is used as the basic component to construct the whole multiplier array. To reduce

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on January 27,2026 at 12:38:41 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Multiple-Precision DPDAC Algorithm

Input: A, B, C (64-bit); Prec(3-bit); Cvt, Para(1-bit);
Output: $D = (\sum_{i=1}^{N=1,2,4} A_i \times B_i + \sum_{j=1}^{M=1,2} C_j)$

Initial: Cnt[31:0] = 32'b0 // Cycle Counter
Repeat: Step 1 ∼ Step 9, Cnt = Cnt + 1
Step 1: Input Data Processing **begin**
  (DP, SP, TF32, HP, BF16) = Decode(Prec[2:0])
  (Sign[3:0], Exp[34:0], Man[52:0]) = Data_Extract(Input_Operands)
end
Step 2: Mantissa Multiplication **begin**
  if $Prec == DP$ **then**
    ManB[52:0] >> 27 if Cnt[0] = 1'b1
    Pd_{DP0}[79:0] = ManA[52:0] × ManB[26:0]
    Pd_{DP}[107:0] + = Pd_{DP0} << 27 if Cnt[0] = 1'b1
  else
    Pd_{SP0}[53:0] = ManA[26:0] × ManB[26:0]
    Pd_{SP1}[51:0] = ManA[52:27] × ManB[52:27]
  end
  {Pd_3[25:0], Pd_2[27:0], Pd_1[27:0], Pd_0[27:0]} = {Pd_{SP1}[51:0], Pd_{SP0}[53:0]}
end
Step 3: Alignment Shifter Count **begin**
  ExpAB_{Max} = Maximum(AB_3, AB_2, AB_1, AB_0)
  ExpC_{Max} = Maximum(C_1, C_0)
  ASC_{Pd_i} = ExpAB_i - ExpAB_{Max} ($i = 0 \sim 3$)
  $\mathrm{ASC_T}_{C_j}$ = ExpAB_{Max} - ExpC_j - (2 × ExpAB_{base} - ExpC_{base}) + (ManC + 3) ($j = 0, 1$)
  ASC_{C_j} = ((ASC_{C_1} and ASC_{C_0}) > 0 and (Para == 0))? $\mathrm{ASC_T}_{C_j}$: (ExpC_{max}- ExpC_j) ($j = 0, 1$)
end
Step 4: Alignment Shifting **begin**
  ShtR_{Pd_i} = Pd_i >> ASC_{Pd_i} // Stage 2
  ShtR_{C_1} = (Zero_{pad}, ManC[162:81]) >> ASC_{C_1}
  Data_{pad} = DP ? ManC[162:81]: Zero
  ShtR_{C_0} = (Data_{pad}, ManC[80:0]) >> ASC_{C_0}
  ShtR_C = {ShtR_{C_1}, ShtR_{C_0}} // for DP FMA
end
Step 5: Dual-Path Addition **begin**
  (Sum_{Pd}, Carry_{Pd}) = CSA4-2(ShtR_{Pd_i}, $i = 0 \sim 3$)
  (A_{Sum}, A_{Carry}) = CSA4-2(Sum_{Pd}, Carry_{Pd}, ShtR_{C_1}, ShtR_{C_0}) // for DPDAC
  Add_h = ShtR_C[162:108] + 1'b1 // for DP FMA
  Add_{l1} = A_{Sum}[107:54] + A_{Carry}[107:54]
  {$\mathrm{A0}_{Sum}$, $\mathrm{A0}_{Carry}$} = Selection({$\mathrm{A0}_{Sum}$, $\mathrm{A0}_{Carry}$},{ShtR_{C_1}, ShtR_{C_0}})
  Add_{l0} = $\mathrm{A0}_{Sum}$[53:0] + $\mathrm{A0}_{Carry}$[53:0] //Reused
  Add_{rslt} = Complement({Add_h, Add_{l1}, Add_{l0}})
end
Step 6 ∼ 8: LZAC, Normalization, Rounding
Step 9: Exponent Processing **begin**
  Exp_1 = Exp- (ExpC_{base} - ExpAB_{base}) //NPCv
  Exp_0 = $\mathrm{ExpC}_{\max}$ - LZC //For Dual-Path
end
Rslt_{Prec} = {Sign, Exp[$W_{\exp}$–1:0], Man[W_{man}-1:0]}

hardware cost, a radix-4 modified Booth multiplier [29] is employed to implement the 14 × 14 multiplication. The 14-bit multiplier operand is padded with 1-bit zero after the LSB and 2-bit zero before the MSB, then the resulting 17-bit vector is divided into eight 3-bit encode groups to generate eight partial products. These eight partial products are merged by two levels of 4-to-2 CSA and a CPA, then generating the product PP_i for $a_i \times b_i$. As shown in Fig. 7(a), every four multipliers are organized into an array to perform $\{a_{i+1}, a_i\} \times \{b_{i+1}, b_i\}$ corresponding to a 28-bit multiplication. Due to the zero padding in the unified vector of mantissa, the product contains variable number of zeros in different modes, as shown in the white region of the products. Fig. 7(b) illustrates the dataflow of the whole multiplier array. Due to the use of iteration methods, DP multiplication is performed every clock cycle with a latency of two cycles, while other lower precision multiplications are implemented in a pipelined manner. A turning signal *Cnt*[0] extracted from the clock counter is used to control the data selection of the input vectors and the results. Only when *Cnt*[0] is "1," the final result is generated in performing DP multiplication. Fig. 7(c) shows the position of the basic 14-bit multiplication to implement the whole 56-bit multiplication. As the proposed multiplier consists of eight basic components, the 16 multiplications are performed in a recursive way over two consecutive clock cycles. The gray block indicates the corresponding multiplication is performed in cycle 0, while the operations in blue block are implemented in cycle 1. The results of these two cycles need to be added to generate the whole result of DP multiplication.

In the design of the multiplier array, the low-power technology is employed to reduce energy consumption. For example, Multiplier 1 and Multiplier 2 can be gated in low-precision multiplications to reduce power consumption, as shown in Fig. 7(a).

C. Alignment Shifter

In both pipeline stage 1 and pipeline stage 2, alignment shifters are required to align the addends and the products, respectively. The alignment shifting method applied in the proposed design is similar to our previous work [26]. In pipeline stage 1, the alignment shifter works in parallel with the mantissa multiplier to reduce the latency. As in a conventional design, the mantissa of C is initially placed 2 bit to the left of the carry save format product, and the maximum alignment amount is reached when the MSB of C is placed 2 bit to the right of the LSB of the product. However, a C-bit carry may be generated in dot-product operations, so the mantissa of C is placed (2+C) bit to the left of the carry save format product in the proposed design, as shown in Fig. 8.

The bitwidth of mantissa C and mantissa A/B is M_q-bit and M_p-bit, respectively. In normal FMA operations, all operands are in the same precision, $M_q = M_p$, while the precision of addend C is higher than that of the multiplier operands A/B in mixed-precision operation, where $M_q > 2 \times M_p$. The shifting operation is controlled by the *ASC*, whose value can be calculated as follows:

$$\mathrm{ASC} = d - \mathrm{const} = (E_c - E_b - E_c + 2p0 - p1) - \mathrm{const}. \quad (9)$$

In (9), the const is introduced based on the initial position offset between *ManC* and *ManA* × *ManB*. To accommodate the shifting requirements of all supported formats, a 163-bit alignment shifter is designed in the proposed design.

In the proposed alignment shifter, the mantissa of addend C is rearranged into a unified format to enable resource sharing among all modes, as shown in Fig. 9. The mantissa of

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on January 27,2026 at 12:38:41 UTC from IEEE Xplore. Restrictions apply.

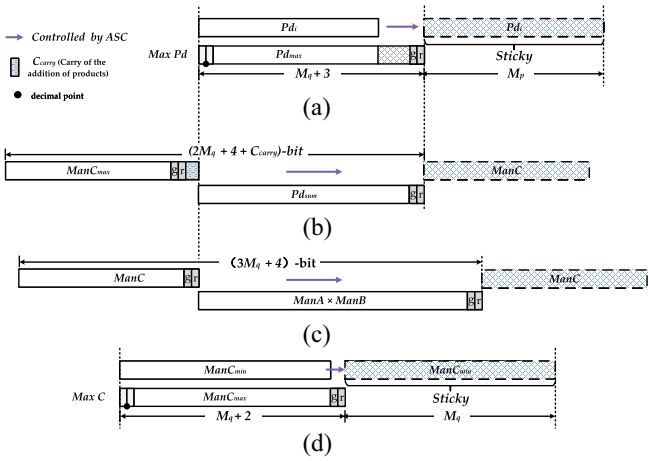


Fig. 8. Alignment for the addends of the proposed DPDAC architecture. (a) Alignment of products in DPAC. (b) Alignment of C operand in DPDAC. (c) Alignment of C operand in standard FMA operation. (d) Alignment of C operands in dual-path execution.

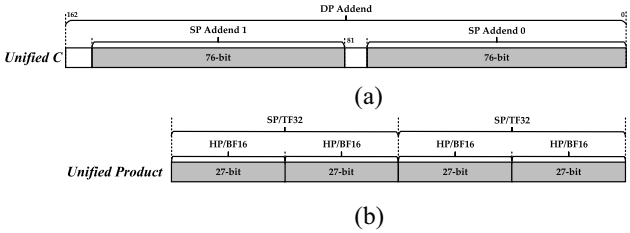


Fig. 9. Unified format of the input vectors of the alignment shifter. (a) Unified addend format of the alignment shifter. (b) Unified product format of the alignment shifter.

each format is extended with zeros to fill the required shifting bitwidth. The unified vectors are right aligned, so that the bit shifted out of the shifter can be used to generate the partial sticky bit. For the alignment of addends, an 8-level alignment shifter is needed to meet the shifting requirement of DP operation. The shifting operations are controlled by *ASC* calculated by the exponent difference module and the control mode signal *Ctrl_M*. If the corresponding bit of *ASCs* is one, right shifting is performed, otherwise the input is selected as the aligned result directly. The *ASC* vector is divided into two parts corresponding to the control of the shifting of two addends C_0 and C_1. In level 0–level 6, two smaller alignment shifters run independently and generate two aligned results for *PD*2_mode and *PD*4_mode. Then, the generated two aligned results are combined, and one 163-bit shifter cell is placed which is controlled by the ASC_3 for *PD*_mode in the last level. Moreover, the shifted out bits in the right of the shifter cell are used to generate the sticky bit, which will be added in the adder.

In 2-term or 4-term DPDAC, two or four products are generated by the multiplier array in pipeline stage 1, respectively. The smaller products are aligned based on the exponent difference of products in pipeline stage 2. As shown in Fig. 10, the exponent difference of products (*Pd*0–*Pd*3) are calculated and four ASCs (*ASC*0–*ASC*3) are generated by using four subtractors. The sign (*S*0, *S*1, and *S*2) of subtractors is used to select the ASCs for products. In 4-term operations, four products need to be added together, and *ASC*0–*ASC*3 are applied

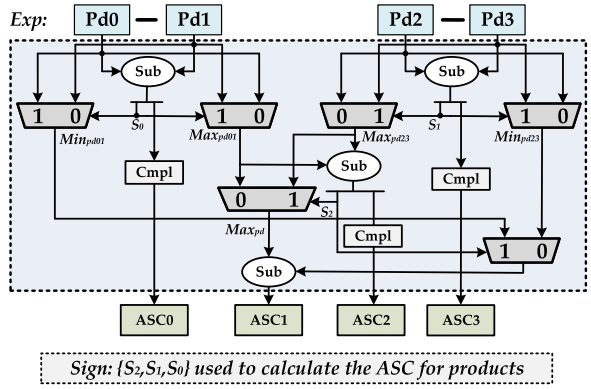


Fig. 10. Calculation of the *ASCs* for the alignment of the products.

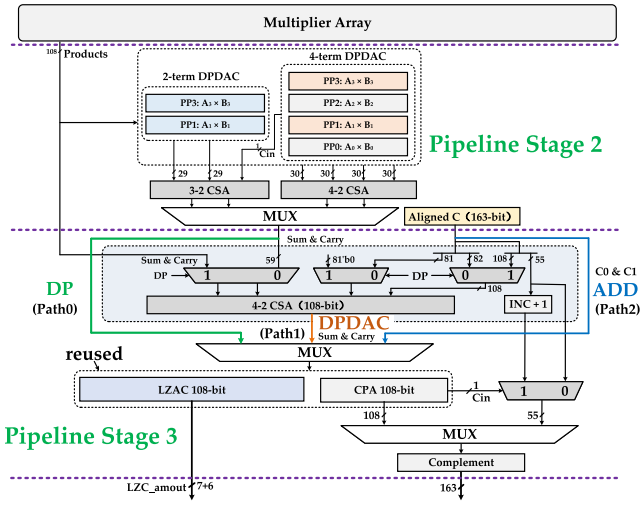


Fig. 11. Datapath of the adder and LZAC.

to control the shifting for the four products. In 2-term operations, only *Pd*0 and *Pd*3 need to be added and the *ASC*2 is used to implement the shifting of the two products.

The bitwidth of the alignment shifter is $M_q + 3$, so a 27 or 56 bit alignment shifter is required for the SP or DP addition, respectively. In the proposed design, four 27-bit alignment shifters are designed in pipeline stage 2, but only three alignment shifters work at any given time due to the maximum product always staying in the initial position. However, compared to using three alignment shifters, our method can save a significant number of multiplexers which are used to select the shifting number and *ASCs* for alignment shifters.

D. Datapath of Addition and Leading Zero Anticipation

The alignment of addend *C* has been performed, and the generated 163-bit vectors need to be added with the products of mantissa multiplication $A \times B$. The datapath of the addition and LZAC are shown in Fig. 11. In pipeline stage 2, the generated products are added by a 4-to-2 CSA and a 3-to-2 CSA. In 4-term DPDAC, only the 4-to-2 CSA works to sum up these four products, while these two CSAs are combined together to perform the addition of the 2-term DPDAC. The carry and sum generated by pipeline stage 1 and stage 2 are selected by the signal *DP*, which is used to enable DP_{mode}. Then,

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on January 27,2026 at 12:38:41 UTC from IEEE Xplore. Restrictions apply.

the selected carry and sum are added together with these two aligned addends C_0 and C_1 by a 4-to-2 CSA. We employed a 163-bit adder to implement the addition of the 163-bit input vector. In *PD*_mode, the addition is split into two parts in pursuit of lower cost. For the most significant 55 bit, an incrementer is applied since only one of the two vectors contains useful data. For the less significant 108 bit, the input vectors need to be added using a CPA. The generated result of the incrementer and the initial input vector at the higher 55 bit are selected by the carry of the lower 108-bit CPA, and combined with the CPA result to generate the addition results. Due to the 163-bit aligned addend being split into two narrower vectors in *PD*2_mode and *PD*4_mode modes, only the lower 108-bit CPA is used and the incrementer is gated to save energy consumption. The sign of the result of the DPDAC architecture is determined by the addition result. Specifically, if the addition result is negative, the sign of addend C is selected and the complement is performed for the addition result. Otherwise, the sign of product $A \times B$ is selected and the addition result is outputted directly.

In order to reduce the latency of the proposed DPDAC datapath, the leading zero anticipation is implemented in parallel with the addition. As both positive results and negative results can be generated by addition, both leading zeros and leading ones need to be anticipated. In this article, the general case indicator presented in [30] is used. The LZAC generates a 7-bit signal *LZC_amount* to perform normalization, but 1-bit error may occur and the modification is needed in the later steps.

E. Normalization and Rounding

The normalization shifter is applied to remove the leading zeros of addition, the architecture of the proposed normalization shifter is similar to the alignment shifter, albeit more simple. Compared to an alignment shifter, the normalization shifting is a left shifting, and the left shifted out bits can be discarded directly. In the normalization shifter, one more final level shifting is added for correcting the error generated by the LZAC. If the leading bit is detected and when it is not one after shifting, 1-bit more left shifting is performed. After shifting, the normalization results are generated and then need to be rounded to generate the mantissa. In the proposed design, roundTiesToEven, the default rounding mode in IEEE 754-2008, is implemented for the supported operations.

Moreover, the *NPCv* is supported in the proposed design, the high-precision results need to be converted to lower precision numbers before outputted to meet the datatype transform requirement of mixed-precision algorithms. Specifically, the conversion operations, DP > SP, SP > TF32, SP > BF16, SP > HP, are required, which are controlled by the input signal *Cvt*. The rounding position is fixed, which is the LSB position of the aimed precision. Therefore, the bits required to perform rounding, the LSB, the rounding bit, and the sticky bit, can be generated easily.

The carry of rounding needs to be added to the LSB position of the aimed formats to generate the final mantissa. If the carry is generated after addition, 1-bit left shifting is performed for mantissa and the exponent should be adjusted to add one.

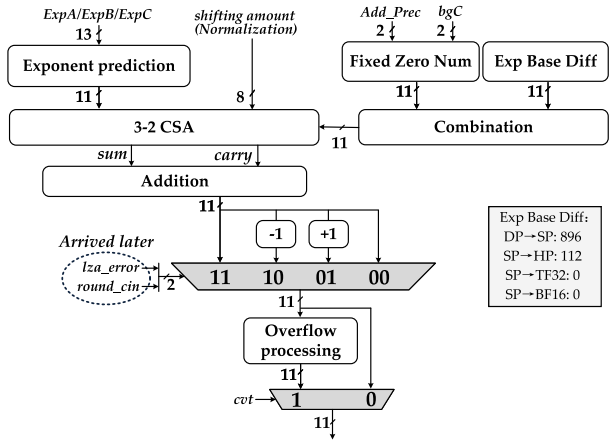


Fig. 12. Exponent processing with the support of *NPCv*.

F. Output Processing

The sign, exponent, and mantissa are processed in the preceding pipeline stages, respectively, and they need to be combined again to generate the final results. However, in order to deal with the subnormal number and *NPCv*, some extra operations are required to process the exponent and mantissa.

For the processing of mantissa, the mantissa is right aligned to the LSB position after rounding, we only need to remove the implicit bit conventionally. For the processing of the exponent, the predicted value of the exponent is calculated in pipeline stage 1. However, the normalization, the rounding carry, and the *NPCv* are not considered, so the predicted exponent needs to be adjusted accordingly.

The exponent processing is performed in parallel with the normalization and rounding to reduce latency, and the datapath is shown in Fig. 12. The shifting amount generated by LZAC and the fixed zero numbers determined by the leading one region need to be considered to obtain the real leading zero numbers. In *PD*_mode, the whole 163-bit adder is used, so the leading one locates in the LSB 108 bit, generating 53 fixed zeros. In *PD*2_mode and *PD*4_mode, no fixed zeros are generated due to only the LSB 108 bit of adder being used. Moreover, the processing for *NPCv* is also implemented. Since the base value of exponent in the supported precisions may differ, the difference between the aimed format and addition format needs to be calculated. The fixed leading zero numbers and the difference of the exponent base value are combined by using a multiplexer. Then, the shifting amount, the predicted exponent, and the combined output generated by the multiplexer are added by using a 3-to-2 CSA and an adder.

Since the shifting amount prediction error *lza_error* detected by normalization and the rounding carry *round_cin* generated after rounding arrive later, the processing for them is performed after addition to avoid a critical path. *lza_error* = 1 means 1-bit more left shifting is performed, so the exponent needs to subtract one, and the *round_cin* = 1 means the carry is generated and adding one is necessary. After *NPCv*, the M-bit wider exponent of the high-precision result has been

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on January 27,2026 at 12:38:41 UTC from IEEE Xplore. Restrictions apply.

TABLE II
DELAY, AREA, AND POWER COMPARISON WITH CONV-DP UNIT

Stage	Area (μm^2)		Power (mW)	
	Prop	Conv-DP	Prop	Conv-DP
Mul & Align	14735	17251	45	49.4
Product Processing	3058	-	11.9	-
Add & LZAC	4301	3173	17.2	16.6
Round & Norm	2015	2220	8.3	12.5
Total	24109	22644	82.4	78.5

TABLE III
FUNCTIONALITIES AND OVERHEAD COMPARISON WITH SINGLE-MODE FP FMA UNITS

	DPDAC	Comb	DP	SP	TF32	HP	BF16
OPs	2-9	2-9	2	2	2	2	2
Ratio (OPs)	1x	1x	1x	2.5x	2.5x	4.5x	4.5x
Area (μm^2)	24109	61586	22644	5283	2451	2376	1981
Percent	100%	255%	94%	22%	10%	10%	8%

processed into an N-bit narrower exponent, and overflow may occur in the processing. In the proposed design, saturation overflow processing is applied in exponent adjustment. If the $(M - N)$-bit of MSB in the exponent is not zeros, the exponent is set to the maximum value that the aimed precision can represent, otherwise the N-bit of LSB is truncated directly.

Finally, the exponent vector and mantissa vector are ORed together and the sign bit is added into the MSB position to form the final FP result.

V. EVALUATION

A. Methodology

The model of the proposed DPDAC architecture is implemented using a structural-level Verilog HDL code. To verify the functionalities, extensive testing vectors are generated with the help of TestFloat [31], and a customized software model is also designed using *C* language to verify the TF32 and BF16 operations. Moreover, we trained the DNN model ResNet-50 [32] in Caffe2 by using GPU V100, and then extracted the intermediate values of some CONV layers. The extracted data can not only be used as the test vectors to verify the correctness but also to evaluate the power consumption for the proposed arithmetic unit. The verification of the proposed design is implemented using SystemVerilog, which can ensure the correctness and provide real-time comparisons for each pipeline stage. For comparison purposes, we also designed single-mode FP FMA units employing conventional architecture for the supported formats.

B. Synthesis Results and Analysis

All designs are synthesized using the 28-nm CMOS technology at the TT process corner, to measure the performance and area, the dynamic power is measured by using the real test vectors with the help of Static Timing Analysis (STA) tools. The proposed DPDAC unit and conventional DP (CONV-DP) FMA unit are synthesized under the worst case delay, 0.68 and 0.60 ns, respectively, their area and power are shown in Table II. Compared to the CONV-DP unit, the proposed design only consumes 6.5% more area and 5.0% more power. The increased overhead is primarily attributed to the pipeline stage 2, which includes the logic for shifting and adding products, as well as the dual-path for enabling parallel execution of DPDAC and addition. Pipeline stage 1 consumes less overhead than the CONV-DP due to the iterative methods employed in the design of the multiplier array. Despite the increased resource requirements of the proposed design in comparison to the CONV-DP implementation, it supports a broad range of formats and a greater variety of operations, such as DPDAC, *NPCv*, and dual-path execution.

C. Comparisons With Single-Mode Discrete Designs

The proposed DPDAC architecture supports multiple formats and various operations for HPC and AI algorithms, providing a more efficient solution than discrete designs that utilize multiple single-mode FP units to achieve the same functionalities. To compare, single-mode FMA units for DP, SP, TF32, and HP formats were designed and synthesized under the same constraints of the proposed design. The architecture of these single-mode units is simpler due to the support of fewer functions or narrower bitwidth of operands, resulting in a smaller area than the proposed one, as shown in Table III. These single-mode FP units can perform one multiplication and one addition per clock cycle, whereas the proposed architecture can support 2–9 operations (OPs) for specific formats. Therefore, multiple single-mode FP units are required to realize the same functionalities. Table III indicates that the combination of these discrete designs consumes 155% more area than the proposed one while being able to realize the same functionalities.

D. Comparisons With Previous Multiple-Precision Works

Table IV presents a comparison of the functionality and performance of the proposed DPDAC with previous works. The proposed DPDAC not only supports DP and SP formats for HPC workloads, but also provides support for reducing formats, such as TF32, BF16, and HP, which are suitable for DNN workloads. Furthermore, the DPDAC architecture enables various types of operations, including standard FMA, dot-product, addition, and dual-path execution. Additionally, the incorporation of the *NPCv* operation in the DPDAC architecture facilitates the conversion of data formats in mixed-precision algorithms.

As shown in Table IV, multiple-precision has been exploited in previous works. The dual-mode designs presented in [13], [14], and [15] support one DP FMA and two parallel SP FMA per clock cycle, which are commonly utilized in traditional HPC systems. However, the utilization of DP and SP operations can incur a significant computational overhead when executing deep learning workloads. In addition to supporting standard FMA operations for DP formats, the designs in [25] and [33] support 2-term dot-product-accumulate (DPAC) for lower formats. The difference is that the 2-term DPAC in [25] is implemented using

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on January 27,2026 at 12:38:41 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
COMPARISON OF FUNCTIONALITY WITH PREVIOUS MULTIPLE-PRECISION FMA DESIGNS

Design	Format	Function	Delay (ns)	Freq (Mhz)	Latency (Cycle)	OP (MACs)	OP TP	Bandwidth $Width(Byte)$	Bandwidth GB/s	Bandwidth $Byte/OP$
ARITH'07 [13]	DP	FMA	3.4	294	3	2	1	24	7.1	12
	SP	FMA	3.4	294	3	4	1	24	7.1	6
MICROE J'08 [33]	DP	FMA	3.61	277	4	2	1	24	6.6	12
	SP	2-term DPAC	3.61	277	4	4	1	24	6.6	12
MICROE J'16 [14]	DP	FMA	2.34	427	3	2	1	24	10.3	12
	SP	FMA	2.34	427	3	4	1	24	10.3	6
arXiv'16 [34]	DP	FMA	0.84	1,190	5	2	1	24	28.6	12
	SP	FMA	0.74	1,360	6	2	1	12	32.6	6
MICPRO'18 [15]	DP	FMA	3.24	308	8	2	1	24	7.4	12
	SP	FMA	3.24	308	8	4	1	24	7.4	6
TC'19 [25]	DP	FMA	1.5	667	3	2	1	24	16.0	12
	SP	2-term DPAC	1.5	667	4	4	1	24	16.0	6
	HP	2-term DPAC	1.5	667	4	8	1	24	16.0	3
TVLSI'20 [35]	DP	FMA	1.08	923	4	2	1	24	22.2	12
	SP	FMA	1.08	923	3	4	1	24	22.2	6
	HP	FMA	1.08	923	3	8	1	24	22.2	3
	BF16	FMA	1.08	923	3	8	1	24	22.2	3
TVLSI'21 [16]	DP	MUL	0.69	1,429	4	1	0.5	40	58.0	40
	SP	5-term DOT-PD	0.69	1,429	4	9	0.5	40	58.0	8.9
	HP	20-term DOT-PD	0.69	1,429	4	39	0.5	40	58.0	2.1
Prop	DP	FMA	0.68	1,471	4	2	0.5	24	35.3	12
	SP	2-term DPDAC	0.68	1,471	4	5	1	24	35.3	4.8
	TF32	2-term DPDAC	0.68	1,471	4	5	1	24	35.3	2.7
	HP	4-term DPDAC	0.68	1,471	4	9	1	24	35.3	2.7
	BF16	4-term DPDAC	0.68	1,471	4	9	1	24	35.3	2.7

Prop-DPDAC: ①single-path: dot-product-dual-accumulate ②dual-path: dot-product and addition in parallel.

mixed-precision methods, while all operands are of the same precision in [33]. The 2-term DPAC is useful in performing digital signal processing applications such as FFTs. Our proposed 4-term DPDAC can be viewed as an extension of the 2-term DPAC, but the need to efficiently sum four terms is significantly more difficult than the 2-term case. FPMax [34] features separate implementations of the FMA operation for DP and SP without any datapath sharing, targeting high-speed ASICs. It is efficient in performing FMA operations for specific formats, but not suitable for HPC and AI algorithms with versatility in data formats. In addition to DP and SP formats, the designs in [16] and [25] support HP format to efficiently accelerate deep learning workloads. The work [16] design an FP dot-product unit for HPC and AI converged computing. It fully exploits the hardware utilization of the multiplier array, then significantly improves the performance of SP and HP operations. However, it requires a higher bandwidth to provide data for the input operands. The proposed one achieves higher $Byte/OP$ than the design in [16] due to the support of additions. Moreover, when using the dot-product unit proposed by [16] to construct a computing system, independent adders are required to sum up the result generated by the dot-product unit, which will significantly increase hardware cost. The design in [35] proposed a multiformat FP architecture supporting both scalar and vector operations for each format. Compared to other multiple-precision designs, it achieves better energy efficiency scaling thanks to the parallel datapaths approach. However, in most HPC and deep learning algorithms, the vector operations in [35] may incur performance loss in performing dot-product operations, such as GEMM, FFT, and CONV.

These previous works are synthesized under different semiconductor technologies. Although the previous work [36] proposed scaling equations for comparing designs across fabrication technologies, but it lacks consideration of the difference of cell library provided by various semiconductor manufacturers. Moreover, the signal activity of the test vectors of these works is also different in power measurement, making it challenging to make a fair comparison between the overhead of the proposed design and that of previous works. However, the architectures in [13], [14], [25], [33], and [34] are also designed by modifying a CONV-DP FMA (baseline design, Base). Hence, a relative comparison that computes the increased overhead on the basis of a CONV-DP FMA is suitable for these designs, as shown in Table V. The designs in [34] optimized to minimize the latency use a

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on January 27,2026 at 12:38:41 UTC from IEEE Xplore. Restrictions apply.

TABLE V
OVERHEAD COMPARISON OF THE PROPOSED DESIGN WITH PREVIOUS WORKS

Design	Area (μm^2 or gates^g)			Power(mW)		
	Base	Prop	Ratio	Base	Prop	Ratio
[13]	600,693	708,590	+18.0%	-	-	-
[33]	35,649^g	38,509^g	+8.0%	-	-	-
[34]	24,000	32,000	+33.3%	41.0	66.0	+61.0%
[14]	515123	672,046	+30.5%	295.0	381.0	+29.2%
[25]	166,160	191,085	+15.0%	39.5	47.3	+19.7%
Prop	22,644	24,109	+6.5%	78.5	82.4	+5.0%

cascade multiply-add with a Wallace tree to sum up the partial products. So it has a shorter latency than the baseline design, but more area and power are needed. The designs in [13], [14], [25], and [33] modify the architecture of a CONV-DP FMA to support multiple-precision operations by sharing the circuit, but they significantly increase the overhead. Compared to prior works, the proposed design supports a wider range of reducing formats (e.g., TF32 and BF16), which are efficient for deep learning tasks and enable additional comprehensive operations, such as DPDAC, *NPCv*, and dual-path execution. Moreover, due to iterative techniques, the proposed design exhibits lower relative overhead compared to a typical DP FMA implementation.

VI. CONCLUSION

In this article, we proposed a DPDAC architecture that can efficiently handle a variety of FP operations, enabling transprecision computing for HPC and AI workloads. The proposed architecture can be configured to perform various operations, such as FMA, addition, dot-product, and so on, by setting the value of operands. The proposed architecture supports both single-path and dual-path execution. In single-path execution, it can perform one standard DP FMA; or 2-term DPDAC for SP and TF32 formats; or 4-term DPDAC for HP and BF16 formats. In dual-path execution, the DPDAC operations are split into two parts that the dot-product and addition, which can be performed in parallel. In addition to supporting multiply-add operations, the DPDAC architecture can be modified to facilitate numerical precision conversion, thereby enabling the convenient conversion of numbers to higher or lower formats to accommodate the datatype transformation in the datapath of mixed-precision algorithms. The proposed DPDAC significantly reduces the overhead compared to the discrete design that utilizes multiple single-mode FP units to achieve the same functionalities. Furthermore, compared to the state-of-the-art multiple-precision designs, the proposed architecture supports a wide range of formats and more types of operations, such as DPAC and *NPCv*, with lower costs. The support of a broad range of formats and more types of operations makes the proposed DPDAC architecture suitable for accelerating both HPC and AI applications.

REFERENCES

[1] G. Yi and V. Loia, "High-performance computing systems and applications for AI," *J. Supercomput.*, vol. 75, no. 8, pp. 4248–4251, 2019.

[2] T. Kurth et al., "Exascale deep learning for climate analytics," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2018, pp. 649–660.

[3] S. Wang, J. Cao, and P. S. Yu, "Deep learning for spatio-temporal data mining: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 8, pp. 3681–3700, Aug. 2022.

[4] A. C. I. Malossi et al., "The transprecision computing paradigm: Concept, design, and applications," in *Proc. Design, Autom. Test Europe Conf. Exhibition (DATE)*, Mar. 2018, pp. 1105–1110.

[5] U. Köster et al., "Flexpoint: An adaptive numerical format for efficient training of deep neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–14.

[6] N. P. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annu. Int. Symp. Comput. Archit.*, 2017, pp. 1–12.

[7] S. Xie et al., "Extreme datacenter specialization for planet-scale computing: ASIC clouds," *ACM SIGOPS Oper. Syst. Rev.*, vol. 52, no. 1, pp. 96–108, 2018.

[8] A. Haidar, S. Tomov, J. Dongarra, and N. J. Higham, "Harnessing GPU tensor cores for fast FP16 arithmetic to speed up mixed-precision iterative refinement solvers," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2018, pp. 603–613.

[9] P. Micikevicius et al., "Mixed precision training," 2017, *arXiv:1710.03740*.

[10] J. Langou, J. Langou, P. Luszczek, J. Kurzak, A. Buttari, and J. Dongarra, "Exploiting the performance of 32 bit floating point arithmetic in obtaining 64 bit accuracy (revisiting iterative refinement for linear systems)," in *Proc. 2006 ACM/IEEE Conf. Supercomput.*, 2006, p. 50.

[11] A. Poulos, S. A. McKee, and J. C. Calhoun, "Posits and the state of numerical representations in the age of exascale and edge computing," *Softw. Pract. Exp.*, vol. 52, no. 2, pp. 619–635, 2022.

[12] D. Kalamkar et al., "A study of BFLOAT16 for deep learning training," 2019, *arXiv:1905.12322*.

[13] L. Huang, L. Shen, K. Dai, and Z. Wang, "A new architecture for multiple-precision floating-point multiply-add fused unit design," in *Proc. 18th IEEE Symp. Comput. Arithmetic (ARITH)*, 2007, pp. 69–76.

[14] K. Manolopoulos, D. Reisis, and V. A. Chouliaras, "An efficient multiple precision floating-point multiply-add fused unit," *Microelectron. J.*, vol. 49, pp. 10–18, Mar. 2016.

[15] V. Arunachalam, A. N. J. Raj, N. Hampannavar, and C. Bidul, "Efficient dual-precision floating-point fused-multiply-add architecture," *Microprocess. Microsyst.*, vol. 57, pp. 23–31, Mar. 2018.

[16] W. Mao et al., "A configurable floating-point multiple-precision processing element for HPC and AI converged computing," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 2, pp. 213–226, Feb. 2022.

[17] J. Sohn and E. E. Swartzlander, "A fused floating-point four-term dot product unit," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 63, no. 3, pp. 370–378, Mar. 2016.

[18] Z. Dan et al., *IEEE Standard for Floating-Point Arithmetic*, IEEE Standard 754-2008, 2008.

[19] H. Liao, J. Tu, J. Xia, and X. Zhou, "DaVinci: A scalable architecture for neural network computing," in *Proc. Hot Chips Symp.*, 2019, pp. 1–44.

[20] T. Chen et al., "DianNao: A small-footprint high-throughput accelerator for ubiquitous machine-learning," *ACM SIGARCH Comput. Archit. News*, vol. 42, no. 1, pp. 269–284, 2014.

[21] Y. Jiao, L. Han, and X. Long, "Hanguang 800 NPU—The ultimate AI inference solution for data centers," in *Proc. IEEE Hot Chips 32 Symp. (HCS)*, 2020, pp. 1–29.

[22] M. Arafa et al., "Cascade lake: Next generation Intel Xeon scalable processor," *IEEE Micro*, vol. 39, no. 2, pp. 29–36, Mar./Apr. 2019.

[23] N. Stephens et al., "The ARM scalable vector extension," *IEEE Micro*, vol. 37, no. 2, pp. 26–39, Mar./Apr. 2017.

[24] M. Baboulin et al., "Accelerating scientific computations with mixed precision algorithms," *Comput. Phys. Commun.*, vol. 180, no. 12, pp. 2526–2533, 2009.

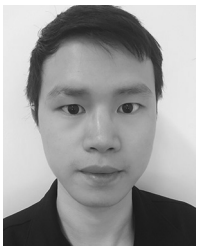
[25] H. Zhang, D. Chen, and S.-B. Ko, "Efficient multiple-precision floating-point fused multiply-add with mixed-precision support," *IEEE Trans. Comput.*, vol. 68, no. 7, pp. 1035–1048, Jul. 2019.

[26] L. Huang, S. Ma, L. Shen, Z. Wang, and N. Xiao, "Low-cost binary128 floating-point FMA unit design with SIMD support," *IEEE Trans. Comput.*, vol. 61, no. 5, pp. 745–751, May 2012.

[27] N. Brunie, F. De Dinechin, and B. De Dinechin, "A mixed-precision fused multiply and add," in *Proc. Conf. Rec. 45th Asilomar Conf. Signals, Syst. Comput. (ASILOMAR)*, 2011, pp. 165–169.

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on January 27,2026 at 12:38:41 UTC from IEEE Xplore. Restrictions apply.

[28] A. Kaivani and S. Ko, "Floating-point butterfly architecture based on binary signed-digit representation," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 24, no. 3, pp. 1208–1211, Mar. 2016.
[29] A. D. Booth, "A signed binary multiplication technique," *Quart. J. Mech. Appl. Math.*, vol. 4, no. 2, pp. 236–240, 1951.
[30] M. S. Schmookler and K. J. Nowka, "Leading zero anticipation and detection-a comparison of methods," in *Proc. 15th IEEE Symp. Comput. Arithmetic*, 2001, pp. 7–12.
[31] J. Hauser. "Berkeley TestFloat." Jan. 2018. [Online]. Available: http://www.jhauser.us/arithmetic/TestFloat.html
[32] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
[33] M. Goek and M. M. Oezbilen, "Multi-functional floating-point MAF designs with dot product support," *Microelectron. J.*, vol. 39, no. 1, pp. 30–43, 2008.
[34] J. Pu, S. Galal, X. Yang, O. Shacham, and M. Horowitz, "FPMax: A 106GFLOPS/W at 217GFLOPS/mm2 single-precision FPU, and a 43.7 GFLOPS/W at 74.6 GFLOPS/mm2 double-precision FPU, in 28nm UTBB FDSOI," 2016, *arXiv:1606.07852*.
[35] S. Mach et al., "FPnew: An open-source multiformat floating-point unit architecture for energy-proportional transprecision computing," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 29, no. 4, pp. 774–787, Apr. 2021.
[36] A. Stillmaker and B. Baas, "Scaling equations for the accurate prediction of CMOS device performance from 180nm to 7nm," *Integration*, vol. 58, pp. 74–81, Jun. 2017.



Hongbing Tan received the B.S. degree from Harbin Engineering University, Harbin, China, in 2015, and the M.S. degree from the National University of Defense Technology, Changsha, China, in 2017, where he is currently pursuing the Ph.D. degree in computer science and technology.

His research interests include computer architecture, and hardware design for deep learning.

Libo Huang received the B.S. and Ph.D. degrees in computer science from the National University of Defense Technology (NUDT), Changsha, China, in 2005 and 2010, respectively.

He was a Professor with NUDT. He has authored over 100 papers in internationally recognized journals and conferences. His current research interests include computer architecture, computer arithmetic, agile hardware design, and big data system.

Zhong Zheng born in 1986. He received the Ph.D. degree in computer science and technology from the National University of Defense Technology (NUDT), Changsha, China, in 2014.

He is currently an Assistant Professor with the College of Computer, NUDT. His main research interests include processor micro-architecture and high-performance computing.

Hui Guo received the M.S. and Ph.D. degrees in computer science and technology from the National University of Defense Technology (NUDT), Changsha, China, in 2013 and 2018, respectively.

He is currently an Assistant Professor with the College of Computer, NUDT. His research interest includes computer architecture, heterogeneous computing, and SIMD architecture.

Qianmin Yang received the B.S., M.S., and Ph.D. degrees in computer science and technology from the National University of Defense Technology (NUDT), Changsha, China, in 2006, 2008, and 2012, respectively.

He is currently an Assistant Professor with the Computer College, NUDT. His main research interests include computer architecture, high-performance computing, and reconfigurable computing.

Li Shen received the Ph.D. degree from the National University of Defense Technology (NUDT), Changsha, China, in 2003.

He is currently a Professor with the School of Computer Science, NUDT. He has published 40 research papers, including IEEE TRANSACTIONS ON COMPUTERS, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, *Micro*, HPCA, and DAC. His main research interests are image super resolution, machine learning, and performance optimization of machine learning systems.

Gang Chen received the B.E. degree in biomedical engineering, the B.S. degree in mathematics and applied mathematics, and the M.S. degree in control science and engineering from Xi'an Jiaotong University, Xi'an, China, in 2008, 2008, and 2011, respectively, and the Ph.D. degree in computer science from the Technical University of Munich, Munich, Germany, in 2016.

He is currently a Professor with Sun Yat-sen University, Guangzhou, China. His research interests include deep learning, computer vision, high-performance computing, and embedded system.

Prof. Chen received the Best Paper Awards on DATE 2021, ICESS 2022, ICET 2021, ESTImedia 2014, and the Best Paper Award Candidate on CODES+ISSS 2020, FPL 2022. He is an Associate Editor of the *Journal of Circuits, Systems, and Computers*.

Liquan Xiao received the M.S. and Ph.D. degrees in computer science and technology from the National University of Defense Technology (NUDT), Changsha, China, in 1994 and 1997, respectively.

He is currently a Professor with NUDT. His research interests include architecture of high-performance computing, high-speed interconnect network, system integration, and power management.

Nong Xiao (Senior Member, IEEE) received the Ph.D. degree in computer science and technology from the National University of Defense Technology, Changsha, China, in 1996.

In 2004, he was a Professor with the Department of Computer Science. He is currently a Professor with the School of Data and Computer Science, Sun Yat-sen University, Guangzhou, China. He has contributed four invited chapters to book volumes and published more than 100 papers in archival journals and refereed conference proceedings. He has developed many grid computing products, e.g. the monitor system of China Grid. His research interests include computer architecture, embedded system, grid computing, and large-scale storage.

Prof. Xiao became a member of ACM in 2004.

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on January 27,2026 at 12:38:41 UTC from IEEE Xplore. Restrictions apply.